《数字图像处理》实验报告八

姓名：史正鑫	学号：226001645	班级：22 计科一班

1. 实验名称： 图像分割

2. 实验内容与目标

1. 加深对图像分割的理解
2. 掌握常用的边缘检测算法
3. 利用 Matlab 实现图像分割

实验任务

1．输入并运行例程，观察边缘检测的效果；
2．在例程基础上，实现图像分割，并试比较 Sobel 边缘检测与 LOG、Canny 边缘检测的差异；
3．撰写实验报告。

3. 实验方法与过程简要介绍（请填入下面空白处）

1.运行例程代码，观察结果

2.实现图像分割，分别使用 Sobel 边缘检测、LOG 边缘检测和 Canny 边缘检测对图像进行处理，观察并比较分割效果。

4. 实验程序与程序注释（请填入下面空白处）

例1 edge 边缘检测 sobel

```
clc
clear

figure;
f = imread('image1.tif');
subplot(231)
imshow(f)
title('原始图像')

[gv1,tvertical] = edge(f,'sobel','vertical');
subplot(232)
imshow(gv1)
title('使用带有自动确定的阈值的一个垂直[sobel]掩模后，函数[edge]导致的结果')

[gv1,thorizontal] = edge(f,'sobel','horizontal');
subplot(233)
imshow(gv1)
title('使用带有自动确定的阈值的一个水平[sobel]掩模后，函数[edge]导致的结果')
```

```
gv2 = edge(f,'sobel',0.15,'vertical');
subplot(234)
imshow(gv2)
title('使用指定阈值的一个垂直[sobel]掩模后，函数[edge]导致的结果')

gboth = edge(f,'sobel',0.15);
subplot(235)
imshow(gboth)
title('使用指定阈值的一个同时考虑水平垂直[sobel]掩模后，函数[edge]导致的结果')

[gboth,tboth] = edge(f,'sobel');
subplot(236)
imshow(gboth)
title('使用自动阈值的一个同时考虑水平垂直[sobel]掩模后，函数[edge]导致的结果')
```

例2 方向边缘检测

```
clc
clear
f = imread('image1.tif');

figure;
subplot(2,2,1)
imshow(f)
title('原始图像')

w45 = [-2 -1 0; -1 0 1; 0 1 2];
g45 = imfilter(double(f),w45,'replicate');
T = 0.3*max(abs(g45(:)));
g45 = g45>=T;
subplot(2,2,2)
imshow(g45)
title('使用函数[imfilter]（具有指定的掩模和阈值）计算[45 度]边缘的结果')

w_45 = [0 1 2; -1 0 1; -2 -1 0];
g_45 = imfilter(double(f),w_45,'replicate');
T = 0.3*max(abs(g_45(:)));
g_45 = g_45>=T;
subplot(2,2,3)
imshow(g_45)
title('使用函数[imfilter]（具有指定的掩模和阈值）计算[-45 度]边缘的结果')

subplot(2,2,4)
imshow(g45 + g_45)
```

```
title('使用函数[imfilter]（具有指定的掩模和阈值）计算[-45 度]+[45 度]边缘的结果')
```

例3 edge 边缘检测 prewitt

```
clc
clear
f = imread('image1.tif');

figure;
subplot(2,2,1)
imshow(f)
title('原始图像')

[gv1,t] = edge(f,'prewitt','vertical');
subplot(2,2,2)
imshow(gv1)
title('使用带有自动确定的阈值的一个垂直[prewitt]掩模后，函数[edge]导致的结果')

gv2 = edge(f,'prewitt',0.15,'vertical');
subplot(2,2,3)
imshow(gv2)
title('使用指定阈值的一个垂直[prewitt]掩模后，函数[edge]导致的结果')

gboth = edge(f,'prewitt',0.15);
subplot(2,2,4)
imshow(gboth)
title('使用指定阈值的一个同时考虑水平垂直[prewitt]掩模后，函数[edge]导致的结果')
```

例4 edge 边缘检测 roberts

```
clc
clear

figure;
f = imread('image1.tif');
subplot(2,2,1)
imshow(f)
title('原始图像')

[gv1,t] = edge(f,'roberts','vertical');
subplot(2,2,2)
imshow(gv1)
title('使用带有自动确定的阈值的一个垂直[roberts]掩模后，函数[edge]导致的结果')

gv2 = edge(f,'roberts',0.15,'vertical');
```

```
subplot(2,2,3)
imshow(gv2)
title('使用指定阈值的一个垂直[roberts]掩模后，函数[edge]导致的结果')

gboth = edge(f,'roberts',0.15);
subplot(2,2,4)
imshow(gboth)
title('使用指定阈值的一个同时考虑水平垂直[roberts]掩模后，函数[edge]导致的结果')
```

%% 例5 LoG, Canny 边缘检测器的比较

```
clc
clear

figure;
f = imread('image1.tif');
subplot(2,3,1)
imshow(f)
title('原始图像')

[g_log_default,tlog] = edge(f,'log');
subplot(2,3,2)
imshow(g_log_default)
title('[LoG]边缘检测器使用默认选项产生的结果')

[g_canny_default,tc] = edge(f,'canny');
subplot(2,3,3)
imshow(g_canny_default)
title('[canny]边缘检测器使用默认选项产生的结果')

[g_log_best,tlog] = edge(f,'log',0.003,2.25);
subplot(2,3,4)
imshow(g_log_best)
title('[LoG]边缘检测器使用 edge(f,log,0.003,2.25) 产生的结果')

[g_canny_best,tc] = edge(f,'canny',[0.04 0.10],1.5);
subplot(2,3,5)
imshow(g_canny_best)
title('[canny]边缘检测器使用 edge(f,canny,[0.04 0.10],1.5) 产生的结果')
```

例6 阈值分割

```
clc
clear
```

```
figure;
f = imread('image2.tif');
subplot(121)
imshow(f)
title('原始图像')
imhist(f)

T = 0.5*(double(min(f(:))) + double(max(f(:))));
done = false;
while ~done
g = f>=T;
Tnext = 0.5*(mean(f(g)) + mean(f(~g)));
done = abs(T - Tnext) < 0.5;
T = Tnext;
end
T
g = f<=T;
subplot(122)
imshow(g)
title('使用迭代方法得到的阈值处理后的图像')
```

例7 区域生长分割

```
% 清屏和清除工作空间
clc;
clear;

% 读取图像
f = imread('image2.tif');

% 显示原始图像
imshow(f);
title('原始图像');

% 显示原始图像的直方图
imhist(f);

% 初始化阈值 T 为图像最小和最大灰度值的平均值
T = 0.5 * (double(min(f(:))) + double(max(f(:))));
done = false;

% 迭代阈值分割
while ~done
% 根据当前阈值 T 分割图像
```

```
g = f >= T;
% 计算新的阈值 Tnext，为分割后的两部分灰度值的平均值
Tnext = 0.5 * (mean(f(g)) + mean(f(~g)));
% 如果阈值变化小于 0.5，则停止迭代
done = abs(T - Tnext) < 0.5;
T = Tnext;
end

% 显示最终确定的阈值
T

% 根据最终阈值分割图像
g = f <= T;
imshow(g);
title('使用迭代方法得到的阈值处理后的图像');

% 清屏和清除工作空间
clc;
clear;

% 读取另一幅图像
f = imread('image1.tif');

% 显示原始图像
imshow(f);
title('原始图像');

% 区域生长算法
[g,NR,SI,TI] = regiongrow(f,100,10);
imshow(SI); % 显示包含种子点的图像
title('种子点图像');

imshow(TI); % 显示包含在经过连通性处理前通过阈值测试的像素
title('所有[经过种子点连通性处理前]通过阈值测试的像素');

% 显示区域生长后的结果
imshow(g);
title('所有通过阈值测试的像素在对种子点进行 8 连通性分析后的结果');


function [g, NR, SI, TI] = regiongrow(f, S, T)
%REGIONGROW Perform segmentation by region growing.
% [G, NR, SI, TI] = REGIONGROW(F, S, T). S can be an array (the
% same size as F) with a 1 at the coordinates of every seed point
```

```
% and 0s elsewhere. S can also be a single seed value. Similarly,
% T can be an array (the same size as F) containing a threshold
% value for each pixel in F. T can also be a scalar, in which case
% it becomes a global threshold. All values in S and T must be in
% the range [0, 1]
%
% G is the result of region growing, with each region labeled by a
% different integer, NR is the number of regions, SI is the final
% seed image used by the algorithm, and TI is the image consisting
% of the pixels in F that satisfied the threshold test, but before
% they were processed for connectivity.
f = tofloat(f);
% If S is a scalar, obtain the seed image.
if numel(S) == 1
SI = f == S;
S1 = S;
else
% S is an array. Eliminate duplicate, connected seed locations
% to reduce the number of loop executions in the following
% sections of code.
SI = bwmorph(S, 'shrink', Inf);
S1 = f(SI); % Array of seed values.
end
TI = false(size(f));
for K = 1:length(S1)
seedvalue = S1(K);
S = abs(f - seedvalue) <= T; % Re-use variable S.
TI = TI | S;
end
% Use function imreconstruct with SI as the marker image to
% obtain the regions corresponding to each seed in S. Function
% bwlabel assigns a different integer to each connected region.
[g, NR] = bwlabel(imreconstruct(SI, TI));
end

function [out, revertclass] = tofloat(in)
identity = @(x) x;
tosingle = @im2single;

table = {'uint8', tosingle, @im2uint8
'uint16', tosingle, @im2uint16
'int16', tosingle, @im2int16
'logical', tosingle, @logical
'double', identity, identity
```

```
'single', identity, identity};

classIndex = find(strcmp(class(in), table(:, 1)));

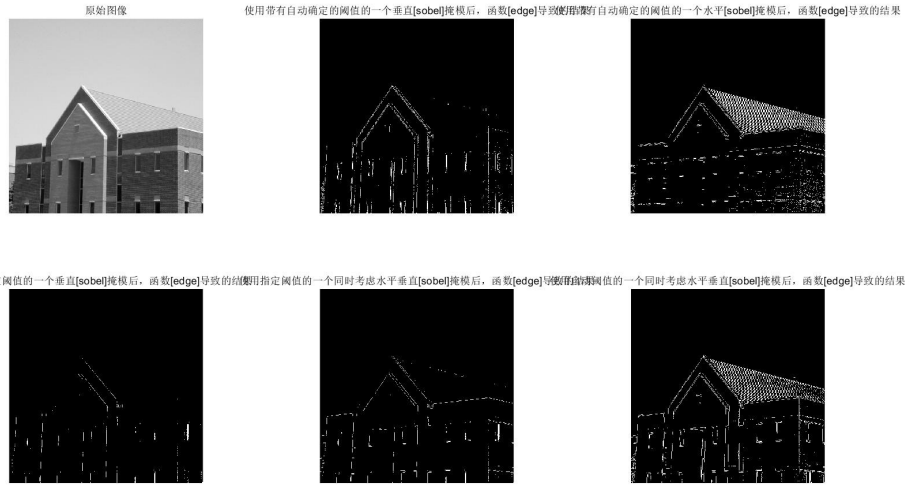
if isempty(classIndex)
error('Unsupported input image class.');
end

out = table{classIndex, 2}(in);

revertclass = table{classIndex, 3};
end
```

5. 实验结果与总结（请填入下面空白处）

（1）

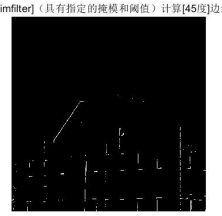


（2）

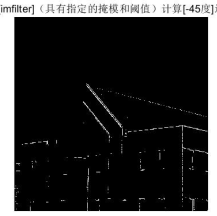
原始图像



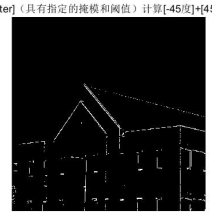
使用函数[imfilter]（具有指定的掩模和阈值）计算[45度]边缘的结果



使用函数[imfilter]（具有指定的掩模和阈值）计算[-45度]边缘的结果



使用函数[imfilter]（具有指定的掩模和阈值）计算[-45度]+[45度]边缘的结果

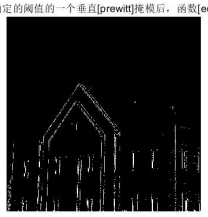


（3）

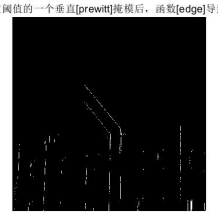
原始图像



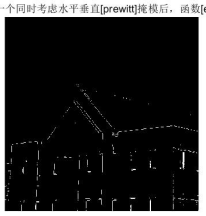
使用带有自动确定的阈值的一个垂直[prewitt]掩模后，函数[edge]导致的结果



使用指定阈值的一个垂直[prewitt]掩模后，函数[edge]导致的结果



使用指定阈值的一个同时考虑水平垂直[prewitt]掩模后，函数[edge]导致的结果

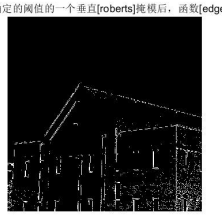


（4）

原始图像



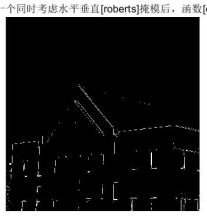
使用带有自动确定的阈值的一个垂直[roberts]掩模后，函数[edge]导致的结果



使用指定阈值的一个垂直[roberts]掩模后，函数[edge]导致的结果



使用指定阈值的一个同时考虑水平垂直[roberts]掩模后，函数[edge]导致的结果



（5）

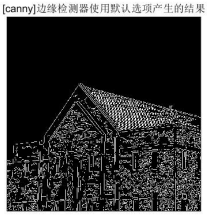
原始图像



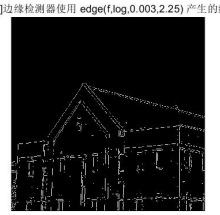
[LoG]边缘检测器使用默认选项产生的结果



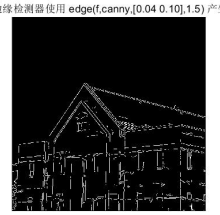
[canny]边缘检测器使用默认选项产生的结果



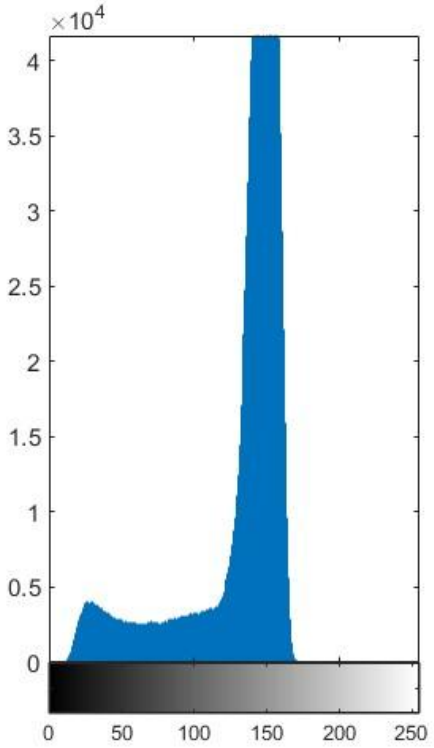
[LoG]边缘检测器使用 edge(f,log,0.003,2.25) 产生的结果



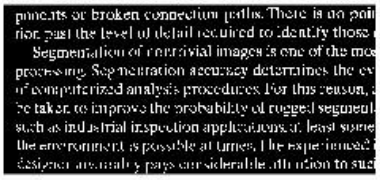
[canny]边缘检测器使用 edge(f,canny,[0.04 0.10],1.5) 产生的结果



（6）



使用迭代方法得到的阈值处理后的图像



（7）

使用迭代方法得到的阈值处理后的图像

ponents or broken connection paths. There is no poi
tion past the level of detail required to identify those
Segmentation of nontrivial images is one of the mos
processing. Segmentation accuracy determines the ev
of computerized analysis procedures. For this reason,
be taken to improve the probability of rugged segment
such as industrial inspection applications, at least some
the environment is possible at times. The experienced
designer invariably pays considerable attention to suc

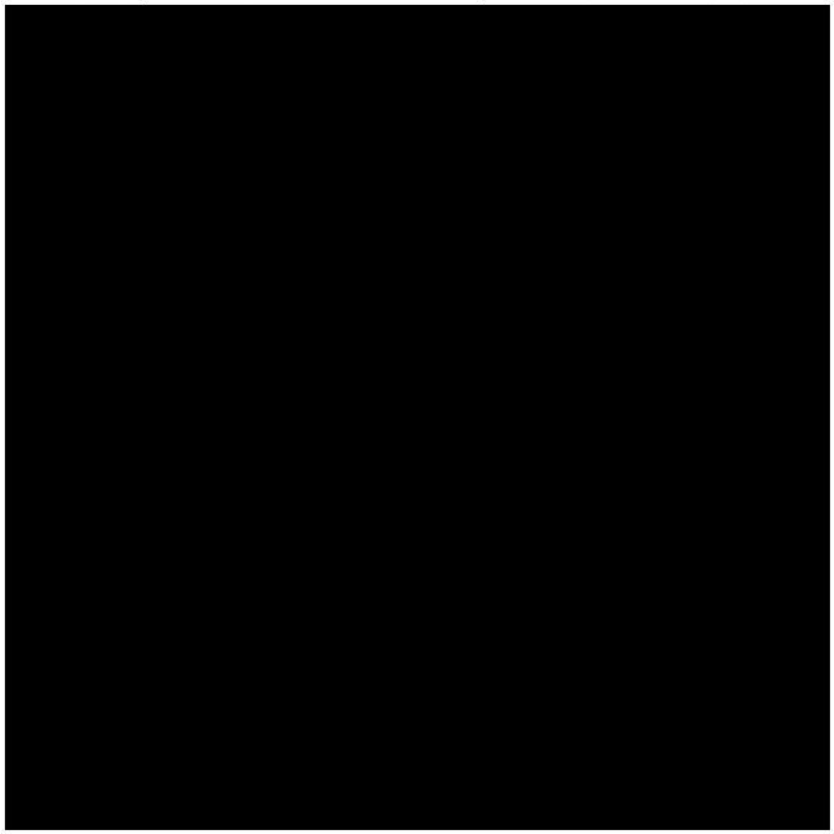
原始图像



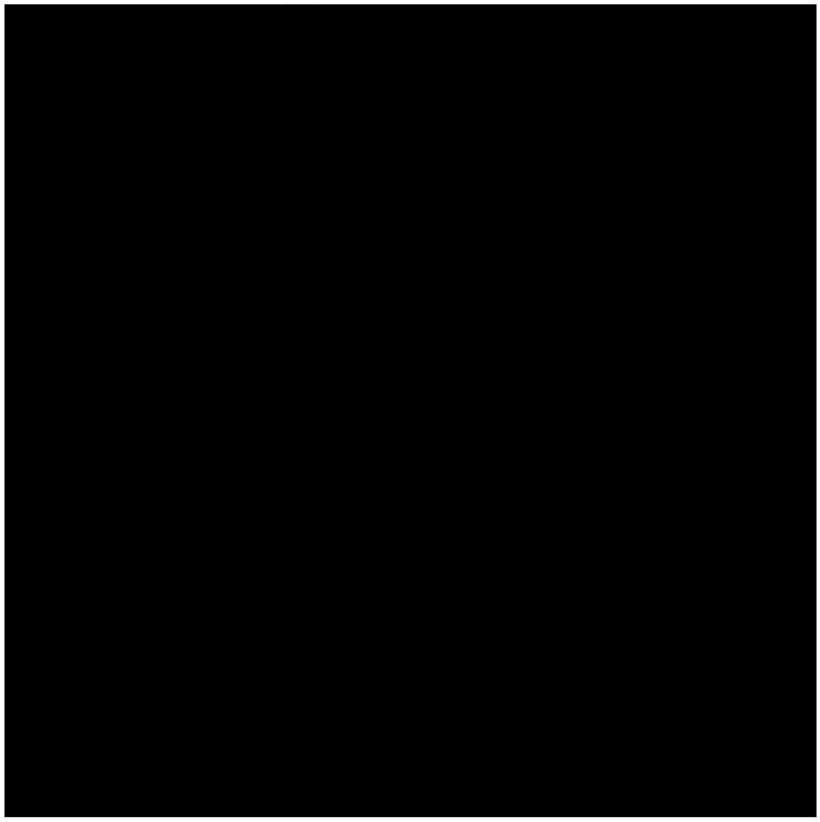
种子点图像



所有[经过种子点连通性处理前]通过阈值测试的像素



所有通过阈值测试的像素在对种子点进行8连通性分析后的结果



实验总结：

在本次实验中，我们深入探讨了边缘检测算法的核心原理，并熟练掌握了 **Sobel**、**LOG** 以及 **Canny** 三种经典边缘检测算法的具体实现。通过实践比较，我们评估了这些算法在图像分割任务中的表现。实验数据表明，**Canny** 算法在边缘检测精度上展现出优越的性能。然而，在现实应用中，选择最适合的边缘检测算法应综合考虑具体需求和实际场景。

此外，我们对迭代阈值分割和区域生长这两种图像处理技术也有了更清晰的认识。迭代阈值分割通过不断迭代计算，精确确定图像分割的阈值，实现图像的有效分割。而区域生长算法则是从选定的种子点出发，按照一定的生长规则扩展区域，直到满足预设条件。这两种技术在图像分割、目标识别等领域具有广泛的应用价值。

在实验过程中，我们也遇到了一些挑战，比如在 **MATLAB** 中无法识别 **regiongrow** 函数。这一问题可能源于我们所使用的 **MATLAB** 版本未包含该函数，或是我们没有正确安装或调用包含该函数的 **MATLAB** 工具箱。

综上所述，本次实验不仅增强了我们对图像处理技术的理解，也锻炼了我们解决实际问题的能力。